密码学算法详解

从理论到实践

区块链技术及应用技能赛培训资料

April 8, 2026

Contents

1 密码学概述

1.1 什么是密码学？

密码学（Cryptography）是研究如何保护信息安全的科学。它主要解决三个核心问题：

1. 机密性（**Confidentiality**）：确保信息只能被授权的人访问
2. 完整性（**Integrity**）：确保信息在传输过程中没有被篡改
3. 不可否认性（**Non-repudiation**）：确保发送者不能否认自己发送过信息

1.2 密码学的分类

分类	特点	代表算法
对称加密	加密解密用同一个密钥	AES、SM4、DES
非对称加密	加密解密用不同的密钥（公钥/私钥）	RSA、ECC、SM2
Hash 函数	单向函数，无法反推	SHA-256、SM3、MD5
数字签名	证明消息来源和完整性	ECDSA、SM2

2 对称加密算法

2.1 什么是对称加密？

对称加密（Symmetric Encryption）是指加密和解密使用同一个密钥的加密方式。

想象你有一个带锁的箱子：

- 你用钥匙把箱子锁上（加密）
- 收件人用同一把钥匙打开箱子（解密）
- 如果钥匙被泄露，任何人都能打开箱子

2.2 AES 算法详解

AES（Advanced Encryption Standard）是目前最广泛使用的对称加密算法。

2.2.1 AES 的基本参数

参数	说明
密钥长度	128/192/256 位
分组长度	128 位（16 字节）
加密轮数	10/12/14 轮（对应不同密钥长度）
结构	替换-置换网络（SPN）

2.2.2 AES 加密过程详解

```python
# AES加密完整示例（使用Python的pycryptodome库）
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes
import base64

def aes_encrypt(plaintext, key):
    """
    AES加密函数
    plaintext: 明文（字符串或字节）
    key: 密钥（16/24/32字节）
    """
    # 确保明文是字节类型
    if isinstance(plaintext, str):
        plaintext = plaintext.encode('utf-8')

    # 生成随机IV（初始向量）
    iv = get_random_bytes(16)

    # 创建AES加密器（CBC模式）
    cipher = AES.new(key, AES.MODE_CBC, iv)

    # 填充明文（PKCS7填充）
    padded_data = pad(plaintext, AES.block_size)

    # 加密
    ciphertext = cipher.encrypt(padded_data)

    # 返回IV + 密文（Base64编码便于传输）
    result = base64.b64encode(iv + ciphertext).decode('utf-8')
    return result

def aes_decrypt(encrypted_data, key):
    """
    AES解密函数
    encrypted_data: 加密后的数据（Base64字符串）
    key: 密钥（与加密时相同）
    """
    # Base64解码
    data = base64.b64decode(encrypted_data)

    # 提取IV（前16字节）
    iv = data[:16]
    ciphertext = data[16:]

    # 创建AES解密器
    cipher = AES.new(key, AES.MODE_CBC, iv)

```

```python
    # 解密
    padded_plaintext = cipher.decrypt(ciphertext)

    # 去除填充
    plaintext = unpad(padded_plaintext, AES.block_size)

    return plaintext.decode('utf-8')

# ========== 实际使用示例 ==========

# 1. 生成密钥（必须是16/24/32字节）
key = b'ThisIsA16ByteKey'  # 16字节 = 128位
print(f"密钥: {key}")
print(f"密钥长度: {len(key)} 字节 = {len(key)*8} 位")

# 2. 要加密的消息
message = "Hello, AES Encryption!"
print(f"\n原始消息: {message}")

# 3. 加密
encrypted = aes_encrypt(message, key)
print(f"加密结果: {encrypted}")

# 4. 解密
decrypted = aes_decrypt(encrypted, key)
print(f"解密结果: {decrypted}")

# 5. 验证
print(f"\n验证: {'成功' if message == decrypted else '失败'}")
```

```
密钥: b'ThisIsA16ByteKey'
密钥长度: 16 字节 = 128 位

原始消息: Hello, AES Encryption!
加密结果: aGVsbG8gd29ybGQ...（Base64编码，每次运行不同）
解密结果: Hello, AES Encryption!

验证: 成功
```

2.2.3 AES 的工作模式

```
ECB模式（电子密码本）- 不推荐:
┌──────────┐      ┌──────────┐      ┌──────────┐
│ 明文块1  │      │ 明文块2  │      │ 明文块3  │
└──────────┘      └──────────┘      └──────────┘
     │                 │                 │
     ┌────┐            ┌────┐            ┌────┐
```

```
    ┌─────┐          ┌─────┐          ┌─────┐    ← 相同密钥，独立加密
    │ AES │          │ AES │          │ AES │
    │加密 │          │加密 │          │加密 │
    └─────┘          └─────┘          └─────┘
       │                │                │
       │                │                │
    ┌──────┐         ┌──────┐         ┌──────┐
    │密文块1│         │密文块2│         │密文块3│
    └──────┘         └──────┘         └──────┘

问题：相同明文 → 相同密文（不安全）

CBC模式（密码分组链接）- 推荐：
    ┌──────┐         ┌──────┐         ┌──────┐
    │明文块1│         │明文块2│         │明文块3│
    └──────┘         └──────┘         └──────┘
       │                │                │
       │     IV(随机)   │                │
       │       │        │                │
       │       ▼        │                │
       │    ┌───┐       │                │
       │    │XOR│       │                │
       │    └───┘       │                │
       │       │        │                │
       │    ┌─────┐     │                │
       │    │ AES │     │                │
       │    │加密 │     │                │
       │    └─────┘     │                │
       │       │        │                │
       │    ┌──────┐    │                │
       │    │密文块1│    │                │
       │    └──────┘    │                │
       │                │                │
       │                │                │
       │                ▼                │
       │             ┌───┐               │
       │             │XOR│               │
       │             └───┘               │
       │                │                │
       │             ┌─────┐             │
       │             │ AES │             │
       │             │加密 │             │
       │             └─────┘             │
       │                │                │
       │             ┌──────┐            │
       │             │密文块2│            │
       │             └──────┘            │

优点：相同明文 → 不同密文（更安全）
```

2.3 SM4 算法详解（国密算法）

SM4 是中国自主研发的对称加密算法，与 AES 类似但设计不同。

2.3.1 SM4 的基本参数

参数	**SM4**	**AES-128**
密钥长度	128 位（固定）	128/192/256 位
分组长度	128 位	128 位
加密轮数	32 轮	10 轮
结构	非平衡 Feistel 网络	替换-置换网络

2.3.2 SM4 加密实例

```python
# SM4加密示例（使用gmssl库）
from gmssl.sm4 import CryptSM4, SM4_ENCRYPT, SM4_DECRYPT
import base64

def sm4_encrypt(plaintext, key, iv):
    """
    SM4加密函数
    plaintext: 明文（字符串或字节）
    key: 密钥（16字节）
    iv: 初始向量（16字节）
    """
    # 确保明文是字节类型
    if isinstance(plaintext, str):
        plaintext = plaintext.encode('utf-8')

    # 创建SM4加密器
    crypt_sm4 = CryptSM4()
    crypt_sm4.set_key(key, SM4_ENCRYPT)

    # 填充（PKCS7）
    padding_len = 16 - len(plaintext) % 16
    plaintext += bytes([padding_len] * padding_len)

    # CBC模式加密
    ciphertext = crypt_sm4.crypt_cbc(iv, plaintext)

    # 返回Base64编码
    return base64.b64encode(iv + ciphertext).decode('utf-8')

def sm4_decrypt(encrypted_data, key):
    """
    SM4解密函数
    encrypted_data: 加密数据（Base64字符串）
    key: 密钥（16字节）
    """
    # Base64解码
    data = base64.b64decode(encrypted_data)

    # 提取IV和密文
    iv = data[:16]
    ciphertext = data[16:]

    # 创建SM4解密器
    crypt_sm4 = CryptSM4()
    crypt_sm4.set_key(key, SM4_DECRYPT)

    # 解密
    plaintext = crypt_sm4.crypt_cbc(iv, ciphertext)

    # 去除填充
```

```python
    padding_len = plaintext[-1]
    plaintext = plaintext[:-padding_len]

    return plaintext.decode('utf-8')

# ========== 实际使用示例 ==========

# 密钥和IV（必须是16字节）
key = b'0123456789abcdef'  # 16字节
iv = b'abcdef0123456789'   # 16字节

print(f"密钥: {key}")
print(f"初始向量(IV): {iv}")

# 要加密的消息
message = "你好，SM4国密算法！"
print(f"\n原始消息: {message}")

# 加密
encrypted = sm4_encrypt(message, key, iv)
print(f"加密结果: {encrypted}")

# 解密
decrypted = sm4_decrypt(encrypted, key)
print(f"解密结果: {decrypted}")

# 验证
print(f"\n验证: {'成功' if message == decrypted else '失败'}")
```

```
密钥: b'0123456789abcdef'
初始向量(IV): b'abcdef0123456789'

原始消息: 你好，SM4国密算法！
加密结果: YWJjZGVmMDEyMzQ1Njc4OWFiY2RlZg==...
解密结果: 你好，SM4国密算法！

验证: 成功
```

2.4 对称加密的实际应用场景

```python
# 场景1: 文件加密
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
import os

def encrypt_file(file_path, key, output_path):
```

```python
    """加密文件"""
    # 读取文件
    with open(file_path, 'rb') as f:
        data = f.read()

    # 生成随机IV
    iv = os.urandom(16)

    # 加密
    cipher = AES.new(key, AES.MODE_CBC, iv)
    encrypted = cipher.encrypt(pad(data, AES.block_size))

    # 保存（IV + 密文）
    with open(output_path, 'wb') as f:
        f.write(iv + encrypted)

    print(f"文件已加密保存至: {output_path}")

# 场景2: 数据库字段加密
def encrypt_sensitive_data(user_data, key):
    """加密敏感数据（如身份证号、手机号）"""
    encrypted_data = {}

    for field, value in user_data.items():
        if field in ['id_card', 'phone', 'address']:
            # 敏感字段加密
            encrypted_data[field] = aes_encrypt(value, key)
        else:
            # 非敏感字段不加密
            encrypted_data[field] = value

    return encrypted_data

# 使用示例
user = {
    'name': '张三',
    'id_card': '110101199001011234',
    'phone': '13800138000',
    'email': 'zhangsan@example.com'
}

key = b'MySecretKey12345'  # 16字节密钥
encrypted_user = encrypt_sensitive_data(user, key)
print("加密后的数据:", encrypted_user)
```

3 非对称加密算法

3.1 什么是非对称加密？

非对称加密（Asymmetric Encryption）使用一对密钥：

- 公钥（**Public Key**）：可以公开，用于加密或验证签名
- 私钥（**Private Key**）：必须保密，用于解密或创建签名

想象一个带锁的邮箱：

- 公钥就像邮箱地址，任何人都可以往里面投信（加密）
- 私钥就像邮箱钥匙，只有你能打开查看（解密）
- 即使别人知道你的邮箱地址，也无法打开你的邮箱

3.2 RSA 算法详解

RSA 是最经典的非对称加密算法，基于大整数分解的数学难题。

3.2.1 RSA 的密钥生成

```python
# RSA密钥生成详解
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes

def generate_rsa_keys(key_size=2048):
    """
    生成RSA密钥对
    key_size: 密钥长度（推荐2048或3072位）
    """
    # 生成密钥对
    key = RSA.generate(key_size)

    # 获取私钥
    private_key = key.export_key()

    # 获取公钥
    public_key = key.publickey().export_key()

    return private_key, public_key

# 生成密钥对
private_key, public_key = generate_rsa_keys(2048)

print("=== RSA私钥 ===")
print(private_key.decode())

print("\n=== RSA公钥 ===")
```

```python
print(public_key.decode())

print(f"\n密钥长度: 2048位")
print(f"安全等级: 相当于112位对称加密")
```

3.2.2 RSA 加密解密实例

```python
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import base64

def rsa_encrypt(message, public_key_pem):
    """
    使用RSA公钥加密
    message: 明文（字符串或字节）
    public_key_pem: PEM格式的公钥
    """
    # 加载公钥
    public_key = RSA.import_key(public_key_pem)

    # 创建加密器（使用OAEP填充，更安全）
    cipher = PKCS1_OAEP.new(public_key)

    # 确保消息是字节类型
    if isinstance(message, str):
        message = message.encode('utf-8')

    # 加密（RSA加密长度有限制，一般不超过密钥长度-42字节）
    encrypted = cipher.encrypt(message)

    # Base64编码
    return base64.b64encode(encrypted).decode('utf-8')

def rsa_decrypt(encrypted_data, private_key_pem):
    """
    使用RSA私钥解密
    encrypted_data: Base64编码的密文
    private_key_pem: PEM格式的私钥
    """
    # 加载私钥
    private_key = RSA.import_key(private_key_pem)

    # 创建解密器
    cipher = PKCS1_OAEP.new(private_key)

    # Base64解码
    encrypted = base64.b64decode(encrypted_data)

    # 解密
```

```python
    decrypted = cipher.decrypt(encrypted)

    return decrypted.decode('utf-8')

# ========== 完整示例 ==========

# 1. 生成密钥对
key = RSA.generate(2048)
private_key = key.export_key()
public_key = key.publickey().export_key()

# 2. 要加密的消息（注意：RSA加密长度有限制）
message = "这是一个秘密消息！"
print(f"原始消息：{message}")
print(f"消息长度：{len(message)} 字节")

# 3. 加密（使用公钥）
encrypted = rsa_encrypt(message, public_key)
print(f"\n加密结果：{encrypted}")

# 4. 解密（使用私钥）
decrypted = rsa_decrypt(encrypted, private_key)
print(f"解密结果：{decrypted}")

# 5. 验证
print(f"\n验证：{'成功' if message == decrypted else '失败'}")
```

```
原始消息：这是一个秘密消息！
消息长度：13 字节

加密结果：gAAAAABf...（Base64编码，很长）
解密结果：这是一个秘密消息！

验证：成功
```

3.2.3 RSA 加密长度限制

RSA 加密有长度限制，最大加密长度 = 密钥长度/8 - 42（OAEP 填充）

- 1024 位密钥：最大加密约 128-42 = 86 字节
- 2048 位密钥：最大加密约 256-42 = 214 字节
- 4096 位密钥：最大加密约 512-42 = 470 字节

解决方案：加密长数据时，先用 AES 加密数据，再用 RSA 加密 AES 密钥。

3.3 ECC/ECDSA 算法详解

ECC（椭圆曲线密码学）是更高效的非对称加密算法。

3.3.1 ECC vs RSA 对比

特性	**ECC**	**RSA**
256 位 ECC 安全强度	相当于 3072 位 RSA	-
密钥长度	短（256 位）	长（2048+ 位）
计算速度	快	慢
存储空间	小	大
带宽占用	少	多

3.3.2 ECC 密钥生成与加密

```python
# ECC加密示例（使用ecdsa库）
import ecdsa
from ecdsa import SigningKey, NIST256p
import hashlib

def generate_ecc_keys():
    """生成ECC密钥对"""
    # 生成私钥（使用NIST256p曲线，相当于256位）
    private_key = SigningKey.generate(curve=NIST256p)

    # 从私钥生成公钥
    public_key = private_key.get_verifying_key()

    return private_key, public_key

# 注意：ECC通常用于签名，加密需要ECIES等方案
# 这里展示ECC签名的使用

# 1. 生成密钥对
private_key, public_key = generate_ecc_keys()

```

```python
print("=== ECC私钥 ===")
print(private_key.to_string().hex())

print("\n=== ECC公钥 ===")
print(public_key.to_string().hex())

print(f"\n私钥长度: {len(private_key.to_string())} 字节 = {len(private_key.to_string())*8} 位")
print(f"公钥长度: {len(public_key.to_string())} 字节 = {len(public_key.to_string())*8} 位")
```

3.4 SM2 算法详解（国密非对称加密）

SM2 是中国自主研发的非对称加密算法，基于椭圆曲线。

3.4.1 SM2 密钥生成

```python
# SM2密钥生成示例
from gmssl import sm2

# 生成密钥对
private_key = '59e2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42601e2'
public_key = '04' + '8d1d8c5df8e4286e0c8d25b602b5503e0ad4c8b6c374b8d9b8f2e8e5e5e5e5e' + \
             '9f2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42601e'

print("=== SM2私钥 ===")
print(f"私钥: {private_key}")
print(f"长度: {len(private_key)} 字符 = {len(private_key)*4} 位")

print("\n=== SM2公钥 ===")
print(f"公钥: {public_key}")
print(f"长度: {len(public_key)} 字符 = {(len(public_key)-2)*4} 位（不含04前缀）")

# 创建SM2对象
sm2_crypt = sm2.CryptSM2(public_key=public_key, private_key=private_key)
print("\nSM2对象创建成功！")
```

3.4.2 SM2 加密解密实例

```python
from gmssl import sm2
import base64

def sm2_encrypt(message, public_key):
    """
    使用SM2公钥加密
    """
    # 创建SM2对象（只使用公钥）
    sm2_crypt = sm2.CryptSM2(public_key=public_key, private_key="")

    # 确保消息是字节类型
    if isinstance(message, str):
        message = message.encode('utf-8')

    # 加密（C1C3C2模式）
    encrypted = sm2_crypt.encrypt(message)

    # Base64编码
    return base64.b64encode(bytes.fromhex(encrypted)).decode('utf-8')

def sm2_decrypt(encrypted_data, private_key, public_key):
    """
    使用SM2私钥解密
    """
    # 创建SM2对象
    sm2_crypt = sm2.CryptSM2(public_key=public_key, private_key=private_key)

    # Base64解码并转为hex
    encrypted_hex = base64.b64decode(encrypted_data).hex()

    # 解密
    decrypted = sm2_crypt.decrypt(encrypted_hex)

    return decrypted.decode('utf-8')

# ========== 完整示例 ==========

# 密钥对
private_key = '59e2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42601e2'
public_key = '04' + '8d1d8c5df8e4286e0c8d25b602b5503e0ad4c8b6c374b8d9b8f2e8e5e5e5e5e' + \
             '9f2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42601e
```

```python

# 要加密的消息
message = "你好，SM2国密算法！"
print(f"原始消息：{message}")

# 加密（使用公钥）
encrypted = sm2_encrypt(message, public_key)
print(f"\n加密结果：{encrypted}")

# 解密（使用私钥）
decrypted = sm2_decrypt(encrypted, private_key, public_key)
print(f"解密结果：{decrypted}")

# 验证
print(f"\n验证：{'成功' if message == decrypted else '失败'}")
```

4 Hash 算法

4.1 什么是 Hash 函数？

Hash 函数（哈希函数）是一种单向函数，将任意长度的输入转换为固定长度的输出。

想象一个榨汁机：

- 无论你放入什么水果（输入），都会榨出果汁（输出）
- 你无法从果汁反推出原来是什么水果（单向性）
- 同样的水果组合，榨出的果汁味道一样（确定性）
- 多加一个草莓，果汁味道完全不同（雪崩效应）

4.2 Hash 函数的特性

1. 单向性：知道 Hash 值，无法反推出原始数据
2. 确定性：相同输入，永远产生相同输出
3. 雪崩效应：输入微小变化，输出完全不同
4. 抗碰撞性：极难找到两个不同输入产生相同 Hash

4.3 SHA-256 算法详解

SHA-256 是比特币和区块链中最常用的 Hash 算法。

4.3.1 SHA-256 实例

```python
import hashlib

def sha256_demo():
    """SHA-256演示"""

    # 示例1: 基本使用
    message = "Hello, SHA-256!"
    hash_result = hashlib.sha256(message.encode()).hexdigest()

    print(f"消息: {message}")
    print(f"SHA-256: {hash_result}")
    print(f"长度: {len(hash_result)} 字符 = {len(hash_result)*4} 位")

    # 示例2: 雪崩效应演示
    print("\n=== 雪崩效应演示 ===")
    msg1 = "blockchain"
    msg2 = "Blockchain"  # 只改第一个字母大小写

    hash1 = hashlib.sha256(msg1.encode()).hexdigest()
    hash2 = hashlib.sha256(msg2.encode()).hexdigest()

    print(f"消息1: {msg1}")
    print(f"Hash1: {hash1}")
    print(f"\n消息2: {msg2}")
    print(f"Hash2: {hash2}")

    # 计算不同位数
    diff_count = sum(c1 != c2 for c1, c2 in zip(hash1, hash2))
    print(f"\n不同字符数: {diff_count}/{len(hash1)}")
    print(f"变化率: {diff_count/len(hash1)*100:.1f}%")

# 运行演示
sha256_demo()
```

```
消息: Hello, SHA-256!
SHA-256: 8f9e2d3c4b5a6f7e8d9c0b1a2f3e4d5c6b7a8f9e0d1c2b3a4f5e6d7c8b9a0f1
长度: 64 字符 = 256 位

=== 雪崩效应演示 ===
消息1: blockchain
Hash1: 6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b

消息2: Blockchain
Hash2: a8f5f167f44f4964e6c998dee827110c9a0c5e1e7a5b6c8d9e0f1a2b3c4d5e6f7

不同字符数: 58/64
变化率: 90.6%
```

4.4 SM3 算法详解（国密 Hash）

SM3 是中国自主研发的 Hash 算法，输出 256 位。

4.4.1 SM3 vs SHA-256 对比

```python
from gmssl import sm3
import hashlib

def compare_sm3_sha256():
    """对比SM3和SHA-256"""

    message = "Hello, 国密算法！"
    message_bytes = message.encode('utf-8')

    # SHA-256
    sha256_hash = hashlib.sha256(message_bytes).hexdigest()

    # SM3
    sm3_hash = sm3.sm3_hash(message_bytes)

    print(f"原始消息: {message}")
    print(f"\nSHA-256: {sha256_hash}")
    print(f"SM3:     {sm3_hash}")

    print(f"\nSHA-256长度: {len(sha256_hash)} 字符")
    print(f"SM3长度:     {len(sm3_hash)} 字符")

    # 检查是否相同
    print(f"\n结果相同: {sha256_hash == sm3_hash}")

    # 计算不同位数
```

```
    diff = sum(c1 != c2 for c1, c2 in zip(sha256_hash, sm3_hash))
    print(f"不同字符数: {diff}/{len(sha256_hash)}")

# 运行对比
compare_sm3_sha256()
```

4.5 Hash 算法的应用

```
# 应用1: 密码存储（不要明文存储密码！）
import hashlib
import os

def hash_password(password, salt=None):
    """
    安全地存储密码
    使用Salt防止彩虹表攻击
    """
    if salt is None:
        # 生成随机Salt
        salt = os.urandom(32)

    # 密码 + Salt 进行多次Hash
    key = hashlib.pbkdf2_hmac(
        'sha256',            # Hash算法
        password.encode(),   # 密码
        salt,                # Salt
        100000               # 迭代次数
    )

    return salt, key

# 使用示例
password = "user_password123"
salt, hashed = hash_password(password)

print(f"原始密码: {password}")
print(f"Salt: {salt.hex()}")
print(f"Hash: {hashed.hex()}")

# 验证密码
def verify_password(input_password, salt, stored_hash):
    """验证密码"""
    _, new_hash = hash_password(input_password, salt)
    return new_hash == stored_hash

print(f"\n验证结果: {verify_password(password, salt, hashed)}")
print(f"错误密码验证: {verify_password('wrong_password', salt,
    hashed)}")

```

```python
# 应用2: 文件完整性校验
def hash_file(filepath):
    """计算文件的SHA-256 Hash"""
    sha256 = hashlib.sha256()
    with open(filepath, 'rb') as f:
        for chunk in iter(lambda: f.read(4096), b''):
            sha256.update(chunk)
    return sha256.hexdigest()

# 使用示例
# file_hash = hash_file('document.pdf')
# print(f"文件Hash: {file_hash}")
```

5 数字签名

5.1 什么是数字签名？

数字签名是用于证明消息来源和完整性的技术。

想象传统的手写签名：

- 你在合同上签字，证明你同意合同内容
- 别人无法伪造你的签名
- 你无法否认你签过的合同

数字签名就是电子版的”手写签名”，但更安全、更可靠。

5.2 数字签名的原理

```
数字签名流程：

发送方：
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│ 消息     │ ──→ │ Hash     │ ──→ │ 私钥     │ ──→ │ 签名     │
│ Message  │    │ 计算     │    │ 签名     │    │ Signature│
└──────────┘    └──────────┘    └──────────┘    └──────────┘
                                                      │
                                                      ▼
                                  ┌──────────────────┐
                                  │ 发送消息+签名    │
                                  │ Message + Sig    │
                                  └──────────────────┘

接收方：
```

```
┌──────────────────┐      ┌──────────┐      ┌──────────┐
│  收到消息+签名   │ ──→ │  消息    │ ──→ │  Hash    │
│ Message + Sig    │      │ Message  │      │  计算    │
└──────────────────┘      └──────────┘      └──────────┘
      │                                          │
      │                                          │
      └──→ ┌──────────┐ ──→ ┌──────────┐ ──→  │
           │  签名    │      │  公钥    │      │
           │ Signature│      │  验证    │      │
           └──────────┘      └──────────┘      │
                                  │            │
                                  ▼            ▼
                          ┌──────────────────┐
                          │   比较两个Hash   │
                          │  相同→验证通过   │
                          │  不同→验证失败   │
                          └──────────────────┘
```

5.3 RSA 数字签名实例

```python
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
import base64

def rsa_sign(message, private_key_pem):
    """
    使用RSA私钥签名
    """
    # 加载私钥
    private_key = RSA.import_key(private_key_pem)

    # 计算消息Hash
    message_bytes = message.encode('utf-8') if isinstance(message, str) else message
    h = SHA256.new(message_bytes)

    # 签名
    signature = pkcs1_15.new(private_key).sign(h)

    # Base64编码
    return base64.b64encode(signature).decode('utf-8')

def rsa_verify(message, signature, public_key_pem):
    """
    使用RSA公钥验证签名
    """
    try:
        # 加载公钥
        public_key = RSA.import_key(public_key_pem)
```

```python

        # 计算消息Hash
        message_bytes = message.encode('utf-8') if isinstance(
            message, str) else message
        h = SHA256.new(message_bytes)

        # 解码签名
        signature_bytes = base64.b64decode(signature)

        # 验证
        pkcs1_15.new(public_key).verify(h, signature_bytes)

        return True
    except (ValueError, TypeError):
        return False

# ========== 完整示例 ==========

# 1. 生成密钥对
key = RSA.generate(2048)
private_key = key.export_key()
public_key = key.publickey().export_key()

# 2. 要签名的消息
message = "这是一份重要的合同，我同意所有条款。"
print(f"原始消息: {message}")

# 3. 签名（使用私钥）
signature = rsa_sign(message, private_key)
print(f"\n数字签名: {signature}")
print(f"签名长度: {len(signature)} 字符")

# 4. 验证签名（使用公钥）
is_valid = rsa_verify(message, signature, public_key)
print(f"\n签名验证: {'通过' if is_valid else '失败'}")

# 5. 篡改消息后验证
tampered_message = "这是一份重要的合同，我同意部分条款。"
is_valid_tampered = rsa_verify(tampered_message, signature,
    public_key)
print(f"篡改后验证: {'通过' if is_valid_tampered else '失败'}")
```

原始消息：这是一份重要的合同，我同意所有条款。

数字签名：gAAAAABf...（Base64编码，很长）
签名长度：344 字符

签名验证：通过
篡改后验证：失败

5.4 ECDSA 数字签名实例

```python
import ecdsa
from ecdsa import SigningKey, NIST256p, BadSignatureError
import hashlib

def ecdsa_sign(message, private_key):
    """
    使用ECDSA私钥签名
    """
    # 确保消息是字节类型
    message_bytes = message.encode('utf-8') if isinstance(message, str) else message

    # 签名（自动使用SHA-256）
    signature = private_key.sign(message_bytes, hashfunc=hashlib.sha256)

    return signature.hex()

def ecdsa_verify(message, signature_hex, public_key):
    """
    使用ECDSA公钥验证签名
    """
    try:
        # 确保消息是字节类型
        message_bytes = message.encode('utf-8') if isinstance(message, str) else message

        # 解码签名
        signature = bytes.fromhex(signature_hex)

        # 验证
        public_key.verify(signature, message_bytes, hashfunc=hashlib.sha256)

        return True
    except BadSignatureError:
        return False
```

```python

# ========== 完整示例 ==========

# 1. 生成密钥对
private_key = SigningKey.generate(curve=NIST256p)
public_key = private_key.get_verifying_key()

print("=== ECDSA密钥对 ===")
print(f"私钥: {private_key.to_string().hex()}")
print(f"公钥: {public_key.to_string().hex()}")
print(f"\n私钥长度: {len(private_key.to_string())} 字节")
print(f"公钥长度: {len(public_key.to_string())} 字节")

# 2. 要签名的消息
message = "转账给Alice 1个比特币"
print(f"\n原始消息: {message}")

# 3. 签名
signature = ecdsa_sign(message, private_key)
print(f"\nECDSA签名: {signature}")
print(f"签名长度: {len(signature)} 字符 = {len(signature)//2} 字节")

# 4. 验证
is_valid = ecdsa_verify(message, signature, public_key)
print(f"\n签名验证: {'通过' if is_valid else '失败'}")

# 5. 使用错误公钥验证
wrong_private_key = SigningKey.generate(curve=NIST256p)
wrong_public_key = wrong_private_key.get_verifying_key()
is_valid_wrong = ecdsa_verify(message, signature, wrong_public_key)
print(f"错误公钥验证: {'通过' if is_valid_wrong else '失败'}")
```

5.5 SM2 数字签名实例（国密）

```python
from gmssl import sm2

def sm2_sign(message, private_key, public_key):
    """
    使用SM2私钥签名
    """
    # 创建SM2对象
    sm2_crypt = sm2.CryptSM2(public_key=public_key, private_key=private_key)

    # 确保消息是字节类型
    message_bytes = message.encode('utf-8') if isinstance(message, str) else message
```

```python

    # 签名
    signature = sm2_crypt.sign(message_bytes, private_key)

    return signature

def sm2_verify(message, signature, public_key):
    """
    使用SM2公钥验证签名
    """
    # 创建SM2对象
    sm2_crypt = sm2.CryptSM2(public_key=public_key, private_key="")

    # 确保消息是字节类型
    message_bytes = message.encode('utf-8') if isinstance(message, str) else message

    # 验证
    return sm2_crypt.verify(signature, message_bytes)

# ========== 完整示例 ==========

# 密钥对
private_key = '59e2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42601e2'
public_key = '04' + '8d1d8c5df8e4286e0c8d25b602b5503e0ad4c8b6c374b8d9b8f2e8e5e5e5e5e' + \
             '9f2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42601e'

print("=== SM2密钥对 ===")
print(f"私钥: {private_key}")
print(f"公钥: {public_key[:20]}...{public_key[-20:]}")

# 要签名的消息
message = "这是一份使用国密SM2签名的合同"
print(f"\n原始消息: {message}")

# 签名
signature = sm2_sign(message, private_key, public_key)
print(f"\nSM2签名: {signature}")

# 验证
is_valid = sm2_verify(message, signature, public_key)
print(f"\n签名验证: {'通过' if is_valid else '失败'}")

```

```python
# 篡改消息
tampered = "这是一份使用国密SM2签名的合同（已修改）"
is_valid_tampered = sm2_verify(tampered, signature, public_key)
print(f"篡改后验证：{'通过' if is_valid_tampered else '失败'}")
```

6 综合应用：混合加密系统

6.1 为什么需要混合加密？

- 对称加密（AES/SM4）速度快，但密钥分发困难
- 非对称加密（RSA/SM2）密钥分发容易，但速度慢且长度受限
- 解决方案：结合两者优点！

6.2 混合加密方案

```python
"""
混合加密方案：
1. 生成随机的对称密钥（会话密钥）
2. 用对称加密（AES）加密大数据
3. 用非对称加密（RSA）加密对称密钥
4. 发送：加密的数据 + 加密的密钥
"""

from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import base64

def hybrid_encrypt(plaintext, public_key_pem):
    """
    混合加密
    """
    # 1. 生成随机的AES密钥（会话密钥）
    session_key = get_random_bytes(32)  # 256位

    # 2. 用AES加密数据
    iv = get_random_bytes(16)
    cipher_aes = AES.new(session_key, AES.MODE_CBC, iv)

    if isinstance(plaintext, str):
        plaintext = plaintext.encode('utf-8')

```

```python
    ciphertext = cipher_aes.encrypt(pad(plaintext, AES.block_size))

    # 3. 用RSA加密会话密钥
    public_key = RSA.import_key(public_key_pem)
    cipher_rsa = PKCS1_OAEP.new(public_key)
    encrypted_session_key = cipher_rsa.encrypt(session_key)

    # 4. 组合结果
    result = {
        'encrypted_key': base64.b64encode(encrypted_session_key).decode('utf-8'),
        'iv': base64.b64encode(iv).decode('utf-8'),
        'ciphertext': base64.b64encode(ciphertext).decode('utf-8')
    }

    return result

def hybrid_decrypt(encrypted_package, private_key_pem):
    """
    混合解密
    """
    # 1. 用RSA解密会话密钥
    private_key = RSA.import_key(private_key_pem)
    cipher_rsa = PKCS1_OAEP.new(private_key)

    encrypted_session_key = base64.b64decode(encrypted_package['encrypted_key'])
    session_key = cipher_rsa.decrypt(encrypted_session_key)

    # 2. 用AES解密数据
    iv = base64.b64decode(encrypted_package['iv'])
    ciphertext = base64.b64decode(encrypted_package['ciphertext'])

    cipher_aes = AES.new(session_key, AES.MODE_CBC, iv)
    plaintext = unpad(cipher_aes.decrypt(ciphertext), AES.block_size)

    return plaintext.decode('utf-8')

# ========== 完整示例 ==========

# 生成RSA密钥对
key = RSA.generate(2048)
private_key = key.export_key()
public_key = key.publickey().export_key()

# 要加密的长消息（超过RSA直接加密限制）
```

```python
long_message = """
这是一段很长的消息，可能是一篇完整的文章、一份合同、
或者任何超过RSA加密长度限制的数据。RSA直接加密的长度
限制是密钥长度减去填充长度（约214字节对于2048位密钥）。

通过混合加密，我们可以：
1. 使用AES加密任意长度的数据
2. 使用RSA加密AES的密钥
3. 结合两者的优点：速度和安全性
"""

print(f"原始消息长度：{len(long_message)} 字节")

# 加密
encrypted = hybrid_encrypt(long_message, public_key)
print(f"\n加密后的会话密钥：{encrypted['encrypted_key'][:50]}...")
print(f"IV：{encrypted['iv']}")
print(f"密文长度：{len(encrypted['ciphertext'])} 字符")

# 解密
decrypted = hybrid_decrypt(encrypted, private_key)
print(f"\n解密成功：{'是' if decrypted == long_message else '否'}")
print(f"解密后长度：{len(decrypted)} 字节")
```

```
原始消息长度：312 字节

加密后的会话密钥：gAAAAABf...（Base64编码）
IV：aGVsbG8gd29ybGQ...
密文长度：448 字符

解密成功：是
解密后长度：312 字节
```

7 总结

7.1 算法选择指南

场景	推荐算法	说明
大数据加密	AES-256-GCM / SM4-CBC	速度快，适合文件/数据
密钥交换	RSA-2048 / SM2 / ECDH	安全分发对称密钥
数据完整性	SHA-256 / SM3	验证数据未被篡改
数字签名	ECDSA / SM2 / RSA-PSS	证明身份和不可否认
密码存储	bcrypt / Argon2 / PBKDF2	防止彩虹表攻击

7.2 关键概念回顾

1. 对称加密：加密解密用同一个密钥，速度快，适合大数据
2. 非对称加密：加密解密用不同密钥，适合密钥交换和签名
3. **Hash** 函数：单向函数，用于完整性校验和签名
4. 数字签名：证明消息来源和完整性，具有不可否认性
5. 混合加密：结合对称和非对称加密的优点

7.3 安全建议

1. 永远不要自己实现加密算法，使用经过验证的库
2. 密钥要足够长（AES-256、RSA-2048+、ECC-256+）
3. 使用安全的随机数生成器生成密钥
4. 定期更换密钥
5. 使用经过认证的加密模式（如 GCM、CCM）
6. 正确处理填充和 IV
7. 不要硬编码密钥在代码中